BetOnMarkets.com

Business Model and System Specification

Note: this document reflects how we wish the system to be, not how it is today.

DRAFT dated May 12, 2013

Contents

Chapter 1

Business synopsis

1.1 Legal structure

Our holding company is **Regent Markets Group Ltd.**, a Jersey company. Jersey is a small British protectorate located just outside of the coast of France. We typically abbreviate this company's name to "RMG".

RMG in turn owns 100% of the following subsidiaries:

- **Regent Markets (Malta) Ltd.** - a company registered in Malta. It is abbreviated to "RMML".
- **Regent Markets (IOM) Ltd.** - a company registered in the Isle of Man. It is abbreviated to "RMIOM".
- **Regent Markets (CR) S.A.** - a company registered in Costa Rica. It is abbreviated to "RMCR".
- **RMG Technology (Malaysia) Sdn Bhd.** - a company registered in Malaysia. It is abbreviated to "RMGTech".

RMML, RMIOM, and RMCR, operate as betting companies, accepting bets on behalf of clients. We call them **Landing Companies**, because betting transactions "land" with either of these companies.

We have decided to land clients and their bets as follows:

- Clients from the UK are landed with RMIOM
- Clients from the rest of Europe are landed with RMML
- Clients from outside of Europe are landed with RMCR

The above is because of regulatory reasons. Indeed, the Isle of Man has a special agreement with the UK, which allows Isle of Man betting companies to freely operate in the UK. That is why UK clients are landed with RMIOM. As

regards European clients, we land them with RMML because Malta is part of Europe and there are freedom of trade and services directives within the EU.

Note: there is an upcoming change in regulation that will happen later this year in Malta, whereby binary options will be moved from betting to financial services regulation.

1.2 How we make money

Our Landing Companies are the counterparties to all bets. This means that it's a zero-sum game between ourselves and our clients: if the client wins a bet, we lose, and vice-versa. We make money by factoring in a small markup on each bet.

We calculate the **theoretical bet price** using mathematical algorithms based on option pricing theory. We then add a small markup (sometimes we call it "commission") to the bet price before offering it to the client.

Note: once a client buys a bet, he can generally re-sell the bet back to the company at prevailing market prices. The general rule is that a bet is available to be re-sold if the **opposite bet** is available for purchase on the website. The opposite bet is the bet that cancels a bet out; for example the opposite of a One Touch bet with strike 100 is a No Touch bet with strike 100, because buying one cancels out the other.

1.3 Broker codes

Clients are identified by their **Login ID**, which is of the form *BBBxxxxx*, where BBB is the **broker code** and xxxxx is a number.

Broker codes are simply codes that identify a group of clients of a given Landing Company. A given Landing Company can have multiple broker codes.

We group clients into broker codes for a variety of reasons. For example, if we create a white-label for a third-party business partner (e.g. the market888.com website which is a partnership with Thierry Delrieu), we might want to assign all the clients of that white label their own broker code. This can help for accounting and reports.

The database server where transactions of clients of a given broker code are recorded is called the **dealing server** for that broker code. A given dealing server can host one or more broker codes. However, all the clients of a given broker code must be on the same dealing server.

For regulatory reasons, the dealing server for a given Landing Company must be physically located in that Landing Company's jurisdiction. Hence, the dealing server for RMML broker codes must be located in Malta, the dealing server for RMIOM broker codes must be located in the Isle of Man, and the dealing server for RMCR broker codes must be located in Costa Rica.

1.4 Client accounts and payments

Clients can fund their accounts and buy bets in a number of currencies. Currently available currencies are the USD, EUR, GBP, and AUD.

We have outsourced our cashier function to Doughflow (`http://www.doughflow.com`), a Costa Rica-based company that specializes in providing cashier functionality to gambling websites. Doughflow integrates with 70+ different payment methods, such as credit card gateways, e-cash wallets, etc. Doughflow provide us with the source code of their system, which we host ourselves on our own (Windows) server. Doughflow do not themselves process the client payment transactions - they only provide us with the software. Hence for example we have our own bank accounts and accounts with credit card processing gateways (such as Datacash), and we input those account numbers and passwords into the Doughflow backoffice interface to enable the given payment method.

Doughflow interface with our system via a RESTful API which we have implemented (in Catalyst/Perl). This is an API that we developed solely for use by Doughflow; it will never be exposed to any other third-party.

1.5 Regulations

Each of our landing companies (RMML, RMIOM, RMCR) is subject to its home country laws and regulations set by the gambling regulator in that country. These regulations generally fall into two categories:

- **AML (anti-money laundering) regulations.** Money laundering is the process by which criminals convert "dirty" money into "clean" money. One way they might try this is by opening accounts with betting firms such as ourselves, and making payments and placing bets in such a way as to obfuscate the source of funds and muddle the trail.
- **KYC (know your client) regulations.** KYC regulations go hand in hand with AML regulations. They are designed to ensure that once certain account thresholds are exceeded, we are required to request identity documents from clients (passport copies, utility bills).

The AML and KYC rules are implemented in our code as **object validations** and are detailed in subsequent chapters of this spec.

1.6 Bets

We offer binary options, also known as binary bets, digital options, digital bets, fixed-odds bets, contingent claims, wagers. These terms are all synonymous. They all however relate to the same thing: a contract whereby the client pays a pre-determined amount (the **stake**) to the company, in exchange for the right to receive from the company a pre-determined amount (the **payout**) upon the occurrence of a certain event in the financial markets.

We offer payouts from $1 to $100,000, and bet **durations** ranging from 30 seconds to 1 year.

There are broadly two types of bets:

- **Non-path-dependent bets (also known as European-style options).** These are bets whose outcome does not depend on the **path** that the underlying takes between the start and end times of the bet. For example, rise/fall bets, higher/lower bets, ends between/ends outside bets.
- **Path-dependent bets (also known as American-style options).** These are bets whose outcome does depend on the path that the underlying takes. For example, One Touch/No Touch bets, Stays Between/Goes Outside bets.

Bets can either one or two **barriers** (also known as strikes). For example the One Touch bet has 1 barrier, whereas the Stays Between bet has 2 barriers (an **upper barrier** and a **lower barrier**).

Each bet has an **opposite bet**. For example the opposite of the One Touch bet is the No Touch bet. A bet and its opposite will cancel each other out.

Selling a bet is equivalent to buying the opposite bet. Therefore, as a general rule, bets can only be sold if the opposite bet is available for purchase.

Bets are priced using a modified version of the Black & Scholes pricing formula. The **theoretical price** of a bet is it's price determined by this formula, without the company markup added. The **Quant team** is responsible for maintaining these formulas.

The company's **monthly turnover** is the aggregate of bet stakes of all bets sold to clients during the month. We use turnover as a measure of how well the business is doing.

1.7 Underlyings and markets

An **underlying** is the instrument upon which a bet is placed; for example the Dow Jones Index, the USD/JPY exchange rate.

We group underlyings into the following categories of **markets**: forex, indices, stocks, commodities, randoms.

"Randoms" are special indices that we generate automatically using random numbers. Surprisingly, betting on random indices is particularly popular with some of our clients.

The current price of an underlying is called it's **spot rate**.

During a given **trading day**, one typically looks at the open, high, low, close (abbreviated to "OHLC") of the underlying.

We obtain data feeds from **feed providers** such as GTIS, Interactive Data, Olsen Data, Telekurs. We store the **ticks** received from the feed providers in the **feed database**, which is also Postgres.

Some times a data feed will contain **stray ticks** (a.k.a. **outliers**) which need to be discarded.

1.8 Trading restrictions

Our trading system is fully automated, therefore we have implemented a number of **trading restrictions** to protect ourselves against abuse. In the code, these are implemented as validations of the Bet object. Please see the relevant chapter of this spec.

1.9 IT

The company's codebase has gone through multiple iterations since inception of the company. Generally speaking, the oldest legacy code is located in /cgi/. A first attempt to refactor the code into objects was performed in 2008-2009, and this code is in /cgi/oop. In 2010 onwards, a major refactoring project was begun, which resulted in the objects located in /lib.

Going forward there is an effort to separate the front-end from the back-end by implementing a RESTful API (the "Web API"), and to re-design the front-end as a Mojolicious application, where Javascript client-side code would call the API to implement the client-side pages.

Client transactions data is stored in a Postgres databases, whereas bet pricing parameters and other system-wide operational data is stored in a distributed CouchDB database.

We do automated testing with Selenium, continuous integration with Jenkins, source control with git, task management with Trello, system monitoring with Zenoss, and soon intend to do configuration management with Chef.

1.10 Marketing

1.10.1 Data & Analytics

The marketing team measures client interaction with the website via Google Analytics, which is implemented via Google Tag Manager.

A lot of traffic is directed to the website via Google Adwords. We measure the **CPA** (Cost per Acquisition of a new account), using the **conversion** tracking functionality of Adwords (again, implemented via Google Tag Manager).

The CPA is compared to the **CLV** (Client Lifetime Value), which is the lifetime value of a client to the company (i.e. amount of profit that the client will generate for the company over the lifetime of his account). Average CLV values are typically calculated per country, as clients from different countries often exhibit very different CLV values (e.g. a client from the UK typically has a much higher CLV than a client from Indonesia).

We also maintain an **OLAP cube**, using the iccube.com system. This OLAP cube is populated from the Postgres database, and is accessed via Excel.

We also maintain reports in the **Jasper** reporting system, which also interfaces to the Postgres database.

1.10.2 Affiliate Program

We offer an affiliate program, whereby we pay **affiliate commissions** to our business partners in exchange for their introducing clients to the website. We use myaffiliates.com as an affiliate platform. Typically we pay affiliates a percentage of the net revenue (i.e. profit) that is generated to the company by their clients, however other affiliate schemes are also available (e.g. percentage of turnover generated).

1.10.3 Free gift/bonus codes

We have a free gift/bonus code system, which allows us to issue **promotional codes** that new clients can input into the account opening form on the website. Promotional codes typically credit the client with a free $10 or $20, however are subject to terms and conditions to prevent abuse.

1.10.4 White labels

We have the ability to create white labels for business partners. The only white label we have at present is market888.com, created for Thierry Delrieu, who markets this site in China.

1.10.5 Other sites

We maintain a blog at blog.betonmarkets.com and a training/informational site at betonmarketsuniversity.com

We also maintain a careers site at regentmarkets.com/careers

1.11 Quants

Quants are tasked with developing the algorithms to price bets. The bet price is derived from the following inputs:

- The bet parameters (barrier(s), expiry date, payout)
- The current price of the underlying (i.e. the spot rate)
- The **volatility surface**
- Interest rates and dividend rates

1.12 Accounts and Finance

The accounts team maintain the monthly management accounts in the xero.com system.

A big task every month is the **reconciliations**, to reconcile the payments (deposits and withdrawals) made by clients between the different systems (i.e.

reconcile between our system and the Doughflow system, between the Doughflow system and the payment gateways, and between the payment gateways and the destination bank accounts).

1.13 Customer Service

Our CS team spans the globe, to provide near-24 hour coverage for clients. We use **desk.com** as a CS platform.

Chapter 2

2013 Goals

2.1 Development of the Web API

This goal is to fully develop the Web API, which will allow (1) fully separating our back-end from our client-side front end, and (2) exposing the Web API to third party business partners and affiliates, thereby creating a B2B business for the company.

The Web API will need to support streaming functionality, notably for the feed part (live streaming of ticks) and the bet pricing part (live streaming of bet prices, for the betting page). The portfolio functionality of the API should also be live streaming.

This live streaming solution (WebSockets?) will replace the legacy Meteor system.

2.2 Mojolicious iteration 2

The Mojolicious iteration 2 project is to port all pages to Mojolicious, with client-side Javascript making calls to the Web API.

2.3 New Charting system

We currently have 4 (!) legacy charting systems: the legacy charts (which draws charts using gnuplot), the Smart Charts (which uses ChartDirector), the interactive charts (an in-house Java applet), and the flash visualization (that appears after buying a short-term bet).

These should all be replaced with a new, state of the art, Javascript charting solution (highstock?), which would in turn grab its data from the Web API feed functionality.

2.4 Integrate Tick trades into main betting interface

Currently the Tick Trades (formerly known as RunBets) have their own separate UI (developed in Flash).

The project here is to deprecate the Flash UI, and to fully integrate Tick Trades into the main betting interface. In other words, just as clients can select bets of day, hourly, minutely, and seconds duration, we will add a "ticks" duration into the betting interface.

In conjunction with this project, we need to refactor the betting page, so that clients remain on the betting page after buying a bet (currently, they are directed to a confirmation page, which interrupts the flow), and any visualizations for short-term bets need to be displayed within the betting page itself.

2.5 Configuration Management of our infrastructure

Our infrastructure goals are (1) scalability, (2) uptime (automated failover, backups, predicting failures) (3) performance (site speed), (4) security.

As a first step towards all these goals, we need to fully roll-out configuration management with Puppet/Chef.

2.6 Other goals

- Refactor legacy code
- Transition off Couch for performance and reliability reasons
- Next generation front-end (asynchronous, windows-based, like the IG Index front-end)
- Mobile apps
- New bet types / "Labs" section of the website

Chapter 3

Components

3.1 Summary

This document is intended to lay out the **business object model** of the Betonmarkets.com business.

Well defined objects allow an elegant WebAPI to be exposed to the outside world.

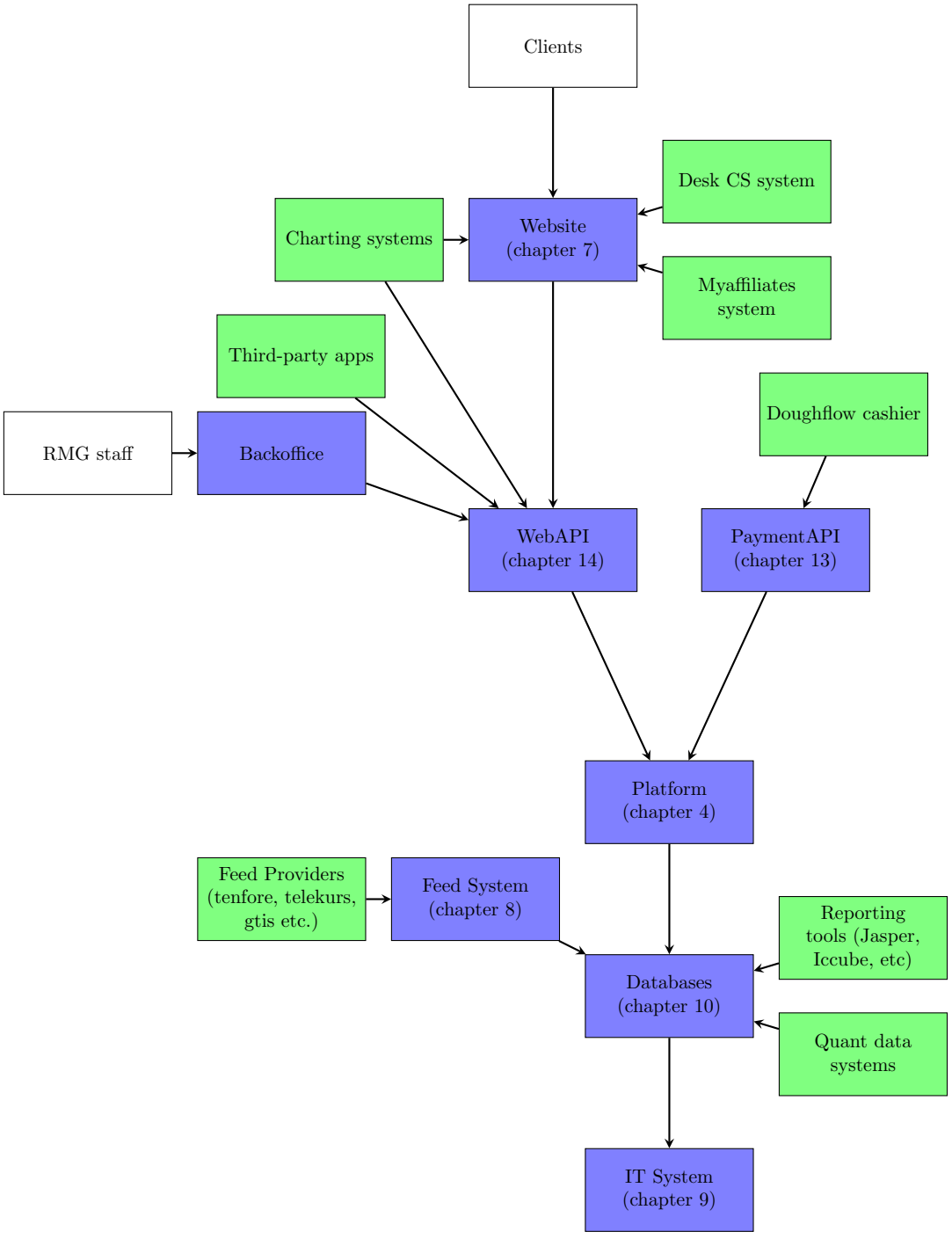
The Betonmarkets.com website and its white-labels are built on top of that WebAPI.

3.2 Diagram

The following diagram shows the **Components** of the system. A Component is a standalone system or service, which conceivably could be managed by a separate IT team, and even commercialized separately.

For example, at some point in the future we might have developers that work exclusively on the Feeds component, on the Website (front-end) component, or on the Platform (back-end system).

In green are third-party components.



Chapter 4

Platform

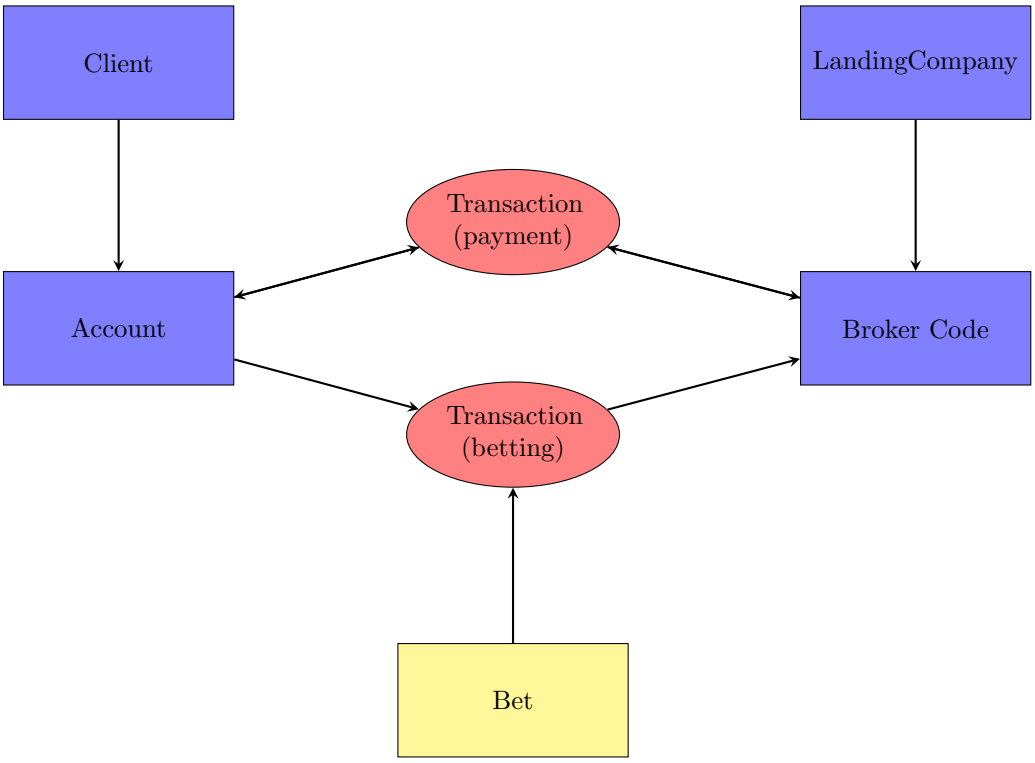
4.1 Summary

Clients have **Accounts**, which can hold multiple currencies. Clients make **Payments** to/from their Accounts (via our outsourced cashier, Doughflow). Clients also make **Transactions** to buy and sell **Bets**.

We have 3 **Landing Companies**: Regent Markets (Malta) Ltd., Regent Markets (IOM) Ltd. (Isle of Man), and Regent Markets (CR) SA (Costa Rica). Each landing company operates under its own set of laws, rules and regulations.

A landing company can have several **Broker Codes**. A broker code allows segregating the clients of a given landing company into distinct groups. For example we might want to make a broker code for a white-label partner.

4.2 Diagram



4.3 Client object

This object represents our clients, who are either individual human beings or (more rarely) corporations.

4.3.1 Attributes

- Attributes of the person: first name, last name, salutation, DOB, email, address, telephone, password, secret question/answer, etc.
- Broker code to which the client is assigned.
- Marketing-related attributes: introducing affiliate, promocode used, etc.
- Status attributes: disabled or not, unwelcome or not, etc.
- Cashier-related attributes: preferred currency, cashier locked or not, etc.
- AML/KYC (anti-money laundering/know-your-client regulations): authenticated or not, authentication details, identity documents.

- Self-exclusion settings.

4.3.2 Validation

- Client must be over 18
- Client must not reside in prohibited countries (USA, Malta, Malaysia, Isle of Man, Hong Kong)
- For a broker code associated with Regent Markets (IOM) Ltd., client residence must be UK or Isle of Man.
- For a broker code associated with Regent Markets (Malta) Ltd., client residence must be within European Union.
- Client email address must be unique (i.e. not used by another client).
- Client mustn't have locked himself out via the self-exclusion facility.
- Client must not be on the UN Terrorists list.
- Client must not be on the disabled list.
- Client name must be unique (This is currently a manual verification performed by CS. It is required as both the IOM and Malta regulators require one account per person.)
- Client IP must be unique for broker code associated with Regent Markets (IOM) Ltd. and Regent Marlkets (Malta) Ltd. (As above, this check is performed manually by CS, and is intended to detect multiple accounts per person.)

Note: currently the Client object in /cgi/oop/ has Validations that should belong instead in the Transaction object. These need to be moved.

Client validation should occur when the client attempts to log in, and when an account is opened.

4.3.3 Client status

To *increase* a client's status, there are these attributes:

- ID AUTHENTICATION - can be either "No ID", or "Pending Authentication" (this is for Costa Rica only - we received the ID scans and need to call them on the phone before moving them to Authenticated statys), or "Authenticated".

- AGE VERIFIED - "Yes" or "No". For IOM, it is automatic via 192. For Costa Rica, we update the field manually during ID authentication. For Malta, some countries work with 192; for the others, we email the client asking for proof of age (and we accept ID scans) - their cashier is locked until then.
- VIP STATUS - we are not using this feature, and will deprecate it.

To *decrease* a client's status, there are these attributes:

- DISABLED - a disabled client will not be able to log into his account.
- UNWELCOME - Clients on this list will be able to log into their accounts, but they will not be able to deposit extra money nor buy any new contracts. They will be able to withdraw money, and they will also be able to close out positions.
- CASHIER LOCKED - a client with a locked cashier won't be able to access the cashier at all.
- WITHDRAWALS LOCKED - only withdrawals are disabled.

4.4 LandingCompany object

The landing company represents one of our three wholly owned subsidiaries: Regent Markets (Malta) Ltd., Regent Markets (IOM) Ltd. (Isle of Man), and Regent Markets (CR) SA (Costa Rica).

4.4.1 Attributes

- Company name
- Company address, phone, fax etc.
- Jurisdiction (Malta, IOM, Costa Rica)
- Allowed currencies
- Associated broker codes
- Regulatory restrictions on types of bets offered

4.5 Account object

Note: the Account object holds only a single currency, hence a Client has several Accounts (one for each currency).

4.5.1 Attributes

- The currency
- The **cash balance** in the given currency.
- A table of **Transaction** objects.

4.5.2 Validation

- Cash balances must be [under XXXX], for each currency individually.

4.6 Broker code object

A broker code can hold multiple currencies.

4.6.1 Attributes

- The **cash balance** in each currency.
- A table of **Transaction** objects.

4.7 Transaction object

A Transaction involves a Client Account, a Broker code, and *either* a Bet or a Payment.

4.7.1 Attributes

- Client account, broker code.
- Bet or Payment
- Timestamp

4.7.2 Validation

- Validations relating to Client Account
  - Client mustn't have used master password to log in
  - Client must have sufficient cash balance
  - Client mustn't be disabled or unwelcome

  - Check whether client passes his self-exclusion limits
  - Check whether client hits daily turnover limits (which are: [....])
  - Check whether client hits weekly turnover limits (which are: [....])
  - All the Client object validations must pass (cf. page 14).
- Validations relating to Broker Code
  - System must be enabled
  - Purchasing bets must be enabled on this broker code
- Validations relating to Bet
  - Bet price mustn't have moved too much
  - If client is selling the bet, check that he owns it on his portfolio
  - All the Bet object validations must pass (cf. page 23)
- Validations relating to Payment
  - All the Payment object validations must pass (cf. page 18)

Chapter 5

Payments

5.1 Summary

There are two types of payments: **deposits** to a Client account, and **withdrawals** to a Client account.

Most payments go through the Doughflow.com outsourced cashier system.

5.2 Payment object

5.2.1 Attributes

- Type of payment (deposit or withdrawal)
- Payment method (bank wire, payment agent transfer, internal transfer, etc)
- Currency
- Amount

5.2.2 Validation - for all payments

- System must be on.
- Cashier must be enabled.
- Client must not have his cashier disabled.
- Client must not have his withdrawals locked (by RMG staff).
- Client must not have locked his cashier himself.

- Check client self-exclusion limits.

5.2.3 Validation - for deposits

- For Regent Markets (IOM) Ltd and Regent Markets (Malta) Ltd client has to be age verified to deposit (except for free bonuses). Exception is for 1st deposit: the deposit will go through, then the age verification process starts, and if it fails, the client's cashier is then locked.
- Deposit amount must be larger than 5 and smaller than 100,000. Note: there are more restrictive limits set in the Doughflow backoffice for each payment method.

5.2.4 Validation - for withdrawals

- Client cash balance must be larger than the withdrawal amount.
- Withdrawal amount must be larger than 5 and smaller than 100,000. Note: there are more restrictive limits set in the Doughflow backoffice for each payment method.
- Client cannot withdraw on the same day that he opened his account.
- Promocode-related withdrawal limits (i.e. terms and conditions of the promocode itself, e.g. turnover of 25 times the promocode amount etc).
- For payment agent withdrawals, don't allow on weekends.
- For Regent Markets (IOM) Ltd clients, aggregate withdrawals of over EUR3000, in 30 days, client must be fully authenticated.
- For Regent Markets (Malta) Ltd, aggregate withdrawals of over EUR2300, client must be fully authenticated.
- For Regent Markets (CR) S.A. withdrawals (i.e. single payment, not aggregate) over USD10,000 require full authentication (this is as per FATF recommendations).

5.2.5 Validation - for payment agent transactions

- Maximum amount per transaction is USD1000.
- Maximum number of transactions per day is 100 payment agent account, 20 client account.
- Maximum transactions per day is USD50,000 payment agent account, USD5,000 client account.

Chapter 6

Product

6.1 Summary

Our products are fixed-odds financial bets - the **Bet** object.

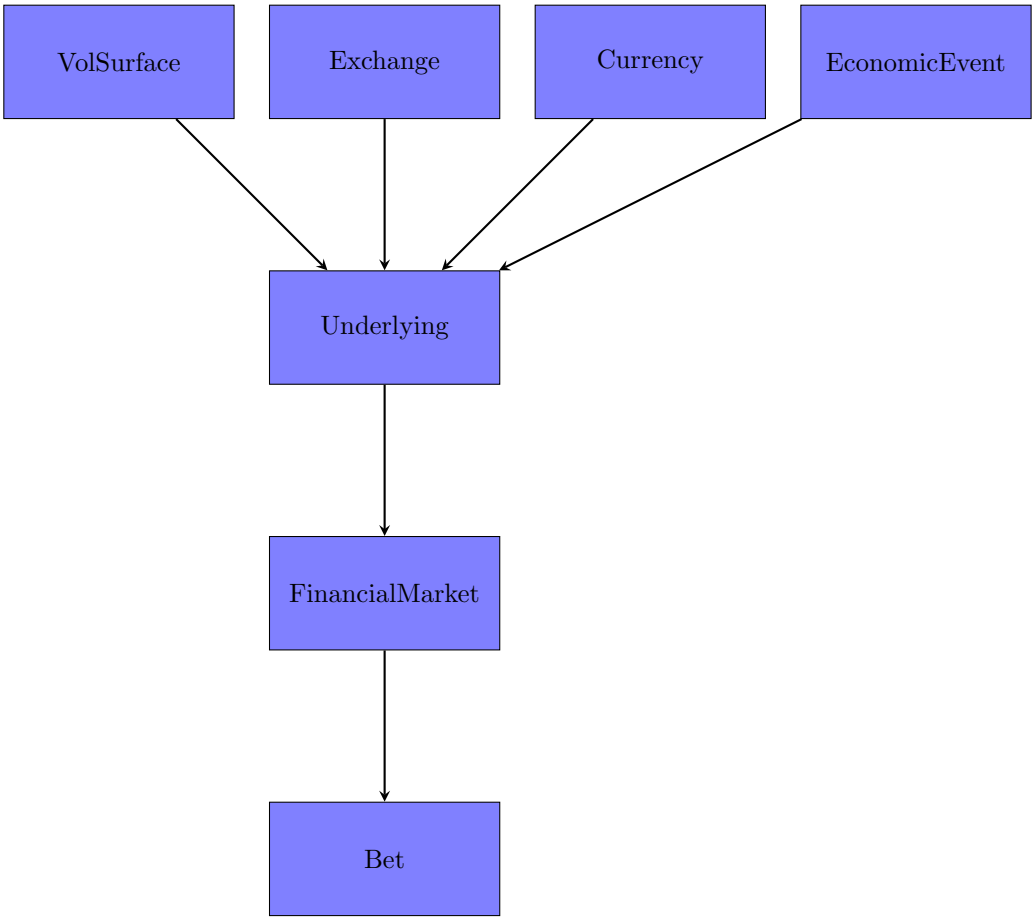
In the future, we might possibly add new products (spread bets, sports bets, casino bets, etc).

To price a Bet, we need to know the bet type, bet parameters (barrier(s), expiry, payout and payout currency), the underlying market, the volatility surface of the underlying market, any economic events that will occur during the bet period, and the pricing engine to use for the pricing.

In general, all the validation checks should be standardized. Currently, we use warn, croak, die, BOM::Exception, etc, all scattered throughout the code. This will be replaced by a Validation Role that can be composed into any object.

6.2 Diagram

- Currently, FinancialMarket is inside of Underlying, I think it should be the other way around, as below.



6.3 Bet object

6.3.1 Attributes

- Required
  - Symbol
  - Bet type
  - Barrier(s)
  - Payout Currency
  - Payout
  - Underlying (notably it's spot value)
  - Date Expiry
  - Date Start

- Optional
  - S is the spot
  - Entry Spot
  - ATM Vols
  - Q Rate
  - R Rate
  - Rho
  - Date Pricing used for BPOT (bet pricing over time)
  - DOMQQQ is the quanto currency data
  - FORDOM is the non-quanto currency data
  - Vol Cut
  - Vol Surface object

6.3.2 Validation (Redesigned in the 'validity' branch)

Note: Severity goes from 1 to 100 (100 is the most severe). We only display one error to the user, the most severe one.

- Validate **Underlying**
  - Global trading shut off
    - All trading must not be suspended on the system
    - Severity: 1
    - Message: Contract is suspended at the moment
  - Trading Suspended on Underlying
    - Trading must not be suspended on the Underlying
    - Severity: 1
    - Message: This contract is not on offer at this time
  - Inefficient Periods
    - Trading must not be suspended for the current time on the Underlying
    - Severity: 1
    - Message: This contract is not on offer at this time
  - Underlying Has Bad Bet Listing
    - Severity: 1
    - Message: Contract is suspended at the moment

- Validate **Quotes**
  - Quote Flags
    - Trading must not be suspended due to fast market conditions Based on code comment, this need to be reviewed
    - Trading must not be suspended due to outliers Based on code comment, this need to be reviewed
    - Soon will only be "Quote Too Old"
    - Severity: 1
    - Message: This contract is not available at the moment because: X
- Validate **Bet type**
  - Suspend Claim Type
    - Trading must not be suspended for the Bet type
    - Severity: 1
    - Message: This contract is not on offer at this time
  - Bet/Underlying Pair Not Permitted
    - Trading must not be suspended for the Bet type, Underlying and Duration (daily or intraday)
    - Severity: 1
    - Message: This contract is not on offer at this time
- Validate **Payout**
  - Non-integer Payout
    - Payout may only contain integers
    - Severity: 1
    - Message: Payout may only contain integers
  - Unknown Payout Currency
    - Payout must be a valid payout currency supported by the broker code (AUD/USD/GBP/EUR)
    - Severity: 1
    - Message: Bad currency (error 2)
  - Payout too high/low
    - Payout must be between 2 and 100,000
    - Severity: 20
    - Message: Payout for this contract must be between 2 and 100000
- Validate **Price**

  - Zero/Undef Bet Price
    * Price must not be zero or empty
    * Severity: 1000
    * Message: We can not properly price your requested contract at this time. Please adjust your contract conditions and try again
  - Price Too Low
    * Price must be greater than 1
    * Severity: 1
    * Message: The price of this contract is less than the minimum price of 1. Please increase the payout or adjust the contract conditions and try again

- Validate **Barrier(s)**
  - Mixed Absolute/Relative Barriers
    * Muliple Barriers must be either absolute, or relative, not mixed together (i.e. not one relative and one absolute)
    * Severity: 5
    * Alert: Yes
    * Message: We could not properly determine the barriers for this contract
  - PD (path-dependent) Barriers Do Not Straddle Spot
    * If Path Dependent, Barrriers must be on either side of the current Spot
    * Severity: 1
    * Message: Your barriers should be on either side of the current spot
  - Relative PD Barriers Not 2 Pips
    * Relative Barriers must be at least 2 pips away from Spot
    * Severity: 1
    * Message: The barrier for this contract must be at least 2 pips away from the spot
  - ATM Bet Type Non-ATM Barrier
    * ATM Barrier must have a zero pip move
    * Severity: 100
    * Alert: Yes
    * Message: The barrier for this contract may not be adjusted
  - Path-Dependent with ATM Entry
    * Path-Dependent bets must have not have zero pip moves

        * Severity: 100
        * Alert: Yes
        * Message: There is an unknown error. Please contact our company with the details of this contract
    - Zero/Negative Absolute Barrier
        * Absolute Barriers must be greater than zero
        * Severity: 100
        * Message: Contract barrier must be positive
    - Barriers Inverted
        * Barriers are switched on the client's behalf
        * Severity: 5
        * Message: The barriers are improperly entered for this contract

- Validate **Start Date**
    - Expiry Before Start
        * Start Date must be before the Expiry Date
        * Severity: 100
        * Message: The start time of the contract must be before the expiry time
    - Start Time in Past
        * Bet cannot start in the past
        * Severity: 100
        * Alert: Yes
        * Message: The contract start time is in the past
    - Exchange not open at Start
        * Exchange must be open when the bet starts
        * Severity: 100
        * Message: The market is currently closed
    - Forward-starting Blackout
        * Minimum forward starting duration is 5 minutes
        * Severity: 100
        * Message: Forward starting contracts must start more than 5 minutes from now
    - Too Close to Open
        * Start Date must be after the blackout time once the market opens, and is defined by market
        * Severity: 1

        - Message: This contract will be available after the first X minutes of the trading session. Presently open Y minutes
    - Too Close to Close
        - Expiry Date must be before the blackout time, and is defined by market Does this do the same thing as the one in Expiry Date?
        - Severity: 1
        - Message: This contract cannot be purchased within the last X minutes of trading
    - Minimum Ticks
        - Minimum Number of Ticks must be greater than or equal to 2 Tick checks to be expanded
        - Severity: 1
        - Message: We can not properly price your requested contract at this time. Please adjust your contract conditions and try again
    - Forward time, Non-FS Bet
        - Forward Starting Date must be before the Start Date for Non Forward Starting bets
        - Severity: 1
        - Message: Input error

- Validate **Expiry Date**
    - Already Expired Bet
        - Cannot purchase a bet that has already expired
        - Severity: 100
        - Alert: Yes
        - Message: Sorry, this contract has already expired
    - Intraday Closed at Expiry
        - Expiry Date must be before the market close
        - Severity: 1
        - Message: We do not allow the purchase of contracts with an expiry after market close. Please adjust the duration and try again
    - Expiration too Close to Close
        - Expiry Date must be before the blackout time, and is defined by market
        - Severity: 1
        - Message: Sorry, contracts cannot expire within the last X minutes of trading

  - Intraday Crosses 0000GMT
    - A bet with duration less than one day must expire within the same day on which it was purchased
    - Severity: 1
    - Message: A contract with duration less than one day must expire within the same day on which it was purchased
  - Can't get OHLC for PD Expiration
    - OHLC is not valid or missing for the current bet
    - Severity: 5
    - Message: There is an unknown error. Please contact our company with the details of this contract
  - No Exit Spot at Expiry
    - Spot at Expiry Time is undefined
    - Severity: 1
    - Message: The database is not yet updated with settlement data
  - Daily Expiry Closed on Expiration Date
    - The requested Expiry Date is on a holiday, early close or weekend
    - Severity: 1
    - Message: The contract must expire on a trading day

- Validate **Duration**
  - Duration Too Long/Short
    - Validate that the duration is greater than the minimum, and less than the maximum allowed
    - Severity: 1
    - Message: The contract is not on offer for the selected duration
  - Short-dated after Vol Rollover
    - Validate that the bet is not purchased between 1700NY and 0000GMT for Forex and Commodities
    - Severity: 1
    - Message: The contract is not on offer for the selected duration
  - Intraday Duration Too Long/Short for the Underlying
    - For Intraday bets, validate that the duration is valid, as defined in the backoffice
    - Severity: 1
    - Message: This contract is not on offer for the selected duration
  - Intraday IV duration Too Long/Short

        * For Intraday bets, validate that the duration is valid, as defined in the backoffice
        * Severity: 1
        * Message: Valid durations for this contract are between X and Y
    - Non-ATM Short Days with Holiday
        * For Non-ATM bets, do not sell bets with less than or equal to 7 days that have a holiday before expiry
        * Severity: 1
        * Message: There is a market holiday during the contract period. Please select an expiry date after X
    - EOY Equities Holiday Blackout
        * For Equities, do not sell bets during the holiday blackout period (end of December, beginning of January)
        * Severity: 1
        * Message: Contract can not expire during the end-of-year holiday period
    - Multiday Duration Too Long/Short
        * Validate that the duration is greater than the minimum, and less than the maximum allowed for a multiday duration
        * Severity: 1
        * Message: This contract is not on offer for the selected duration
    - Too Low Trading/Holiday Ratio
        * Ensure that there are not too many holidays
        * Severity: 1
        * Message: Too many market holidays during the contract period
- Validate **Vol Surface**
    - Vol Surace errors
        * Validate that the smiles are arbitrage free (i.e. no smile flags errors)
        * For Forex, validate that the surface is less than 4 hours old
        * Validate that the surface date is after the trade date
        * Severity: 100
        * Alert: Yes
        * Message: The system is missing critical pricing information. Please try again in a few minutes
- Validate **Time to Sell**
    - Sale Right After Purchase

        * Validate that one minute has passed (or two minutes during quiet periods) before allowing sell back
        * Severity: 1
        * Message: We can not properly price your requested contract at this time. Please adjust your contract conditions and try again
    - Sale of Intraday Bet
        * Validate that intraday bets cannot be sold back (unless it is on Randoms)
        * Severity: 1
        * Message: Sale of contracts with less than 1 day remaining is not permitted

- Validatations re. **Intraday Historical** pricer
    - IH Delta Out of Range
        * For intraday bets, barrier deltas must be between 0.15 and 0.85
        * Severity: 1
        * Message: The barrier entered was not within the acceptable range
    - IH Not Enough Ticks
        * sub-2 case creates fake ticks to allow to proceed
        * Severity: 1
        * Message: This contract is not on offer at this time due to current market conditions
    - IH Missing Seasonality
        * Seasonality must be defined for the current underlying
        * Severity: 1
        * Message: This contract is not on offer at this time due to current market conditions

- Validate **Legacy Bet**
    - Legacy Bet
        * Expires at 0 value
        * Severity: 1
        * Message: This contract is no longer offered

Note: to validate a bet sale, we simply validate the purchase of the opposite bet.

6.4 Underlying object

6.4.1 Attributes

- Underlying Symbol
- Bets available (dailydoubles, endsinout, flashs, etc)
- Display Name ('AUD/USD')
- Inverted indicator (i.e. AUD/USD = 0 or USD/AUD = 1)
- Market (forex / indices / commodities / random)
- Market Convention (atm setting, bf, delta premium adjusted, delta style, rr)
- Quanto Only indicator
- Pip Size
- Spot Spread (0.0002 for AUDUSD)
- Spot Spread Size (2 for AUDUSD)
- Tick Interval (number of seconds * 5 to indicate feed down)
- Opening hours
- Delay amount (i.e. quote display licensing terms)
- Tick interval (if applicable, how many seconds between two consecutive ticks)
- Spot delta market convention
- Display decimals (how many decimals to display)

6.4.2 Validation

- Validation on Underlying
  - Underlying must not be disabled
- Validation on Underlying
  - Underlying must be open for trading
- Validation on Underlying

  - We shouldn't be within a prohibited trading time (first XX minutes of trading session, last XX minutes of trading session)
- Validation on Spot
  - Spot must be within sanity bounds
- Validation on Spot
  - Fast market check
- Validation on Underlying Symbol
  - Underlying Symbol must be specified
- Validation on Underlying Symbol
  - Underlying Symbol must exist in yml file
- Validation on Underlying Symbol
  - Underlying Symbol must be less than 15 characters in length
- Validation on Bets
  - Verify bet categories are valid
- Validation on Pip Size
  - Verify Pip Size exists This check needs review, as suggested by the code comment
- Validation on Currency
  - Verify that Quoted Currency is defined Should this be handled by the currency object?
- Validation on Full Feed File Date
  - Verify that Date is Valid
- Validation on Start Date and End Date
  - Verify that Start Date is before the End Date

6.5 VolSurface object

6.5.1 Attributes

- Surface type (by delta, by moneyness)

- Symbol (i.e. frxAUDUSD)
- Recorded date
- Cutoff (for FX surfaces - i.e. NY10am etc)
- Available tenors
- Interpolation Method (i.e. solve quadratic)
- Surface
- Default Spread is 0.1
- Effective Date in epochs
- Print Precision is the number of decimal places to show, currently 4 for AUDUSD
- Smile Points (i.e. 25, 50, 75 for Delta Surfaces)

6.5.2 Validation in BOM::Volatility::SurfaceValidator

- Validation of Surface (Currently being redesigned)
  - Check if Spot Reference exists for Moneyness Surfaces
  - Check if Age of Surface is less than 6 hours
  - Check Structure
    - Verify at least two smiles per surface
    - Verify Tenor Days are non-negative
    - Verify Vol Points are non-negative
    - Verify Maximum tenor is less than 380 days for Delta Surfaces, and 750 Days for Monyness Surfaces
    - Verify Smiles have at least two points
    - Verify that ON smile exists for Forex
    - Verify that the Difference between two smile points are less than 30 vol points for FX and 100 vol points for Indices
    - Verify that the Smiles have the same number of points
  - Check Smiles and Admissible Checks
    - Verify Monotonicity of Smiles
    - Verify Convexity of Smiles
  - Term Structure Check for Calendar Arbitrage

6.6 Exchange object

6.6.1 Attributes

- Symbol (i.e. FOREX)
- Display Name (i.e. Forex)
- Market Times has Early Closes dates, and Standard Open, Close and Settlement Times
- Delay Amount, most often 0
- Holidays
- Offered indicator
- Open On Weekends indicator
- Tenfore Trading Timezone (i.e. UTC)
- Trading Timezone (i.e. UTC)

6.6.2 Validation

- Validation on Symbol
  - Symbol must be specified
- Validation on Holidays
  - Sends email if last Holiday is before today
- Validation on Open and Close Times
  - Warn if Open or Close Times are missing
- Validation on Total Trading Time, throw error if less than zero

6.7 FinancialMarket object

6.7.1 Attributes

- Name (i.e. forex)
- Display Name (i.e. Forex)
- Reduced Display Decimals set to 1 for forex

- Vol Cut Off set to 'NY1000' for forex Should perhaps move to Underlying?
- Asset Type (i.e. currency)
- Bets Offered indicator
- Display Order for forex on is 1
- Absolute Barrier Multiplier most likely 1

6.7.2 Validation

- We pull these attributes directly from app config or financial markets.yml with out validation.
- For example, we pull disabled directly from app config without checking if the value is 0 or 1.

6.8 Currency object

6.8.1 Attributes

- Symbol (i.e. AUD)
- Rates is the interest rate scedule
- For Date is the date in question for the desired rate

6.8.2 Validation

- Validation for Rates and Date
  - Checks if both Rates and Date are both defined
- This section could use more checks

6.9 EconomicEvent object

6.9.1 Attributes

- Symbol
- Release Date

- Recorded Date
- Impact
- Source
- Event Name

6.9.2 Validation

- Validation on Release Date
  - Check that Release Date does not predate Recorded Date
- Validation input Params
  - Params must be passed as hashref

Chapter 7

Website

7.1 Summary

Clients log in to a given **Website**, which is either betonmarkets.com or a white-label thereof. A **Session** is created, which is maintained by the Mojolicious framework.

Note: the code has a legacy concept of "skins", which we are deprecating. A "skin" and a "website" will become the same thing.

7.2 Website object

7.2.1 Attributes

- Primary URL (e.g. "betonmarkets.com")
- Broker codes that may log into this website
- Available languages

7.3 BOM::Web::Controller object

This is a Mojolicious object. It maintains a session, in which we can store:

- Client who logged in
- Language being displayed
- Website
- Selected currency
- Selected payout
- Selected underlying

Note: this code is in the bom-web git repo.

Chapter 8

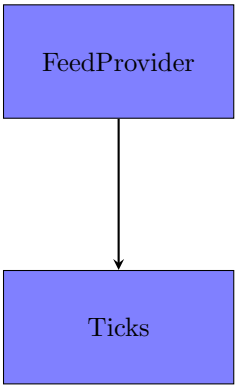
Feed System

8.1 Summary

We receive **data feeds** from various **feed providers** (Tenfore, Telekurs, GTIS, Olsen).

Data feeds are a realtime stream of **ticks**.

8.2 Diagram



8.3 FeedProvider object

8.3.1 Attributes

- Name
- Connection script
-

8.4 Tick object

8.4.1 Attributes

- Timestamp
- Quote
-

8.4.2 Validation

- Timestamp must be realtime (i.e. not more than XX seconds old)
- Check if not outlier/stray tick
-

Chapter 9

IT System

9.1 Environments

An **Environment** is a group of servers upon which a website can run. Here are example Environments: production, production-market888, development, staging [to be deleted], qa01, qa02.

TODO: move environments to /config/files/environment/

9.2 Server naming conventions

Naming conventions for database servers:

- Dealing Database server instance: [landingcompany shortcode]-dbpri[DD] - e.g. cr-dbpri01
- Mirror Database (binary replication) is cr-dbpri02
- Mirror Database (bucardo replication) is cr-dbsec01

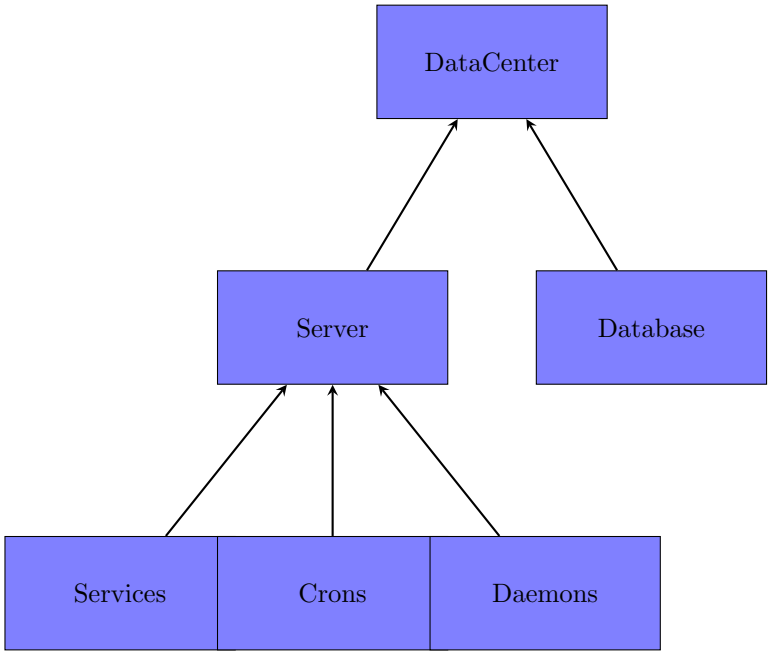
Naming conventions for host servers: [landingcompany shortcode]-host[DD] - e.g. cr-host01 Note: For host servers not attached to a landing company, we call them lo-host01 etc.

Naming convention for openvz web servers: www01 (the load balancer being www).

Naming convention for openvz api servers: api01 (the load balancer being api).

Naming convention for feed servers: feed01

9.3 Diagram



9.4 Server object

9.4.1 Attributes

-

9.4.2 Validation

-

Chapter 10

Databases

10.1 CouchDB

The CouchDB databases are:

Product related:

- economic_events
- volatility_surfaces
- interest_rates
- dividends

Website related:

- bom (contains the appconfig website settings)

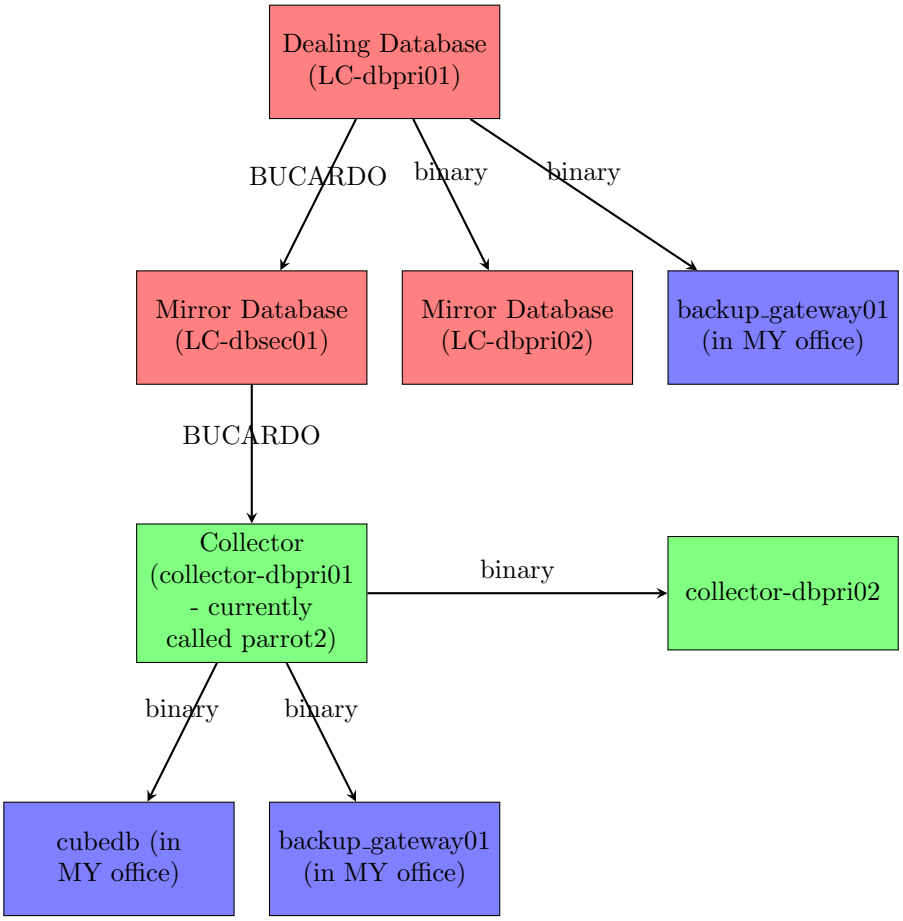
The master Couch server is parrot1. Only parrot1 writes to Couch.

10.2 Postgres

10.2.1 Client and transactions database

A **Landing Company** can own one or more **dealing databases** (which are named in postgres "**regentmarkets**"). For example, Regent Markets (CR) SA owns two dealing databases (we made a separate one for the market888.com clients).

Dealing databases are replicated as follows:



Notes:

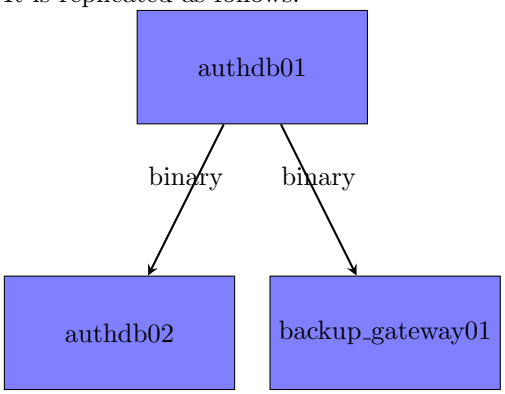
- Bucardo replication is used to **Collector**, since it's the only way that Collector can collect data from multiple dealing databases.
- backup_gateway01 is a write-only backup of all our postgres data (we never read from it).
- General rule: every database that we have has a binary replication in the same datacenter (datacenters have the same color in the diagram above).
- The naming convention for dealing database servers is LC-dbpriXX where LC is the Landing Company short code, and XX is a number. For example, cr-dbpri01.

10.2.2 Authentications database

This database (called "**authdb**" in postgres) is for application definitions and client one-time passwords and sessions (i.e. for session management and the

WebAPI).

It is replicated as follows:

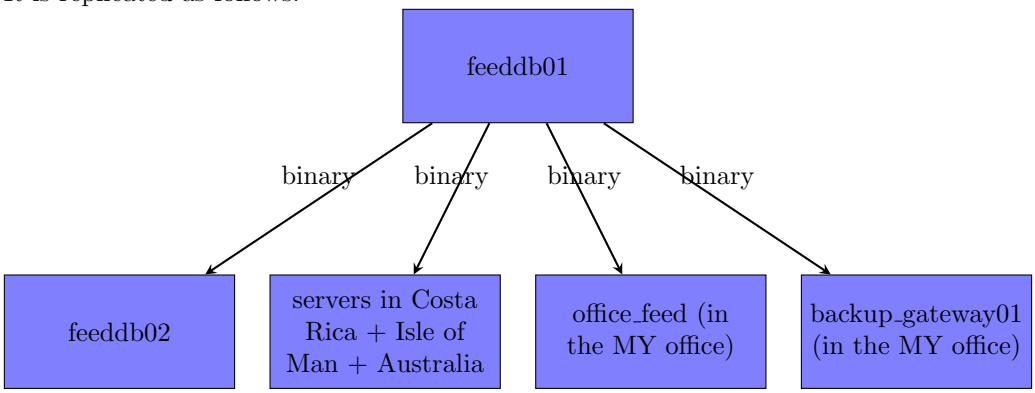


Note: there are Redis caches on each server to cache the authentication tokens retrieved from authdb01.

10.2.3 Feed database

The feed database contains OHLC (daily open, high, low, close market data) and tick-by-tick market data.

It is replicated as follows:



We replicate to servers in Costa Rica, Isle of Man, and Australia, to give fast access to servers close to those geographic areas.

We replicate to office_feed for use by developers.

We replicate to backup_gateway01 because that is write-only collector for all our databases.

10.2.4 Tipster database

This is a mysql database, which needs to be migrated to postgres in due course.

Chapter 11

Monitoring

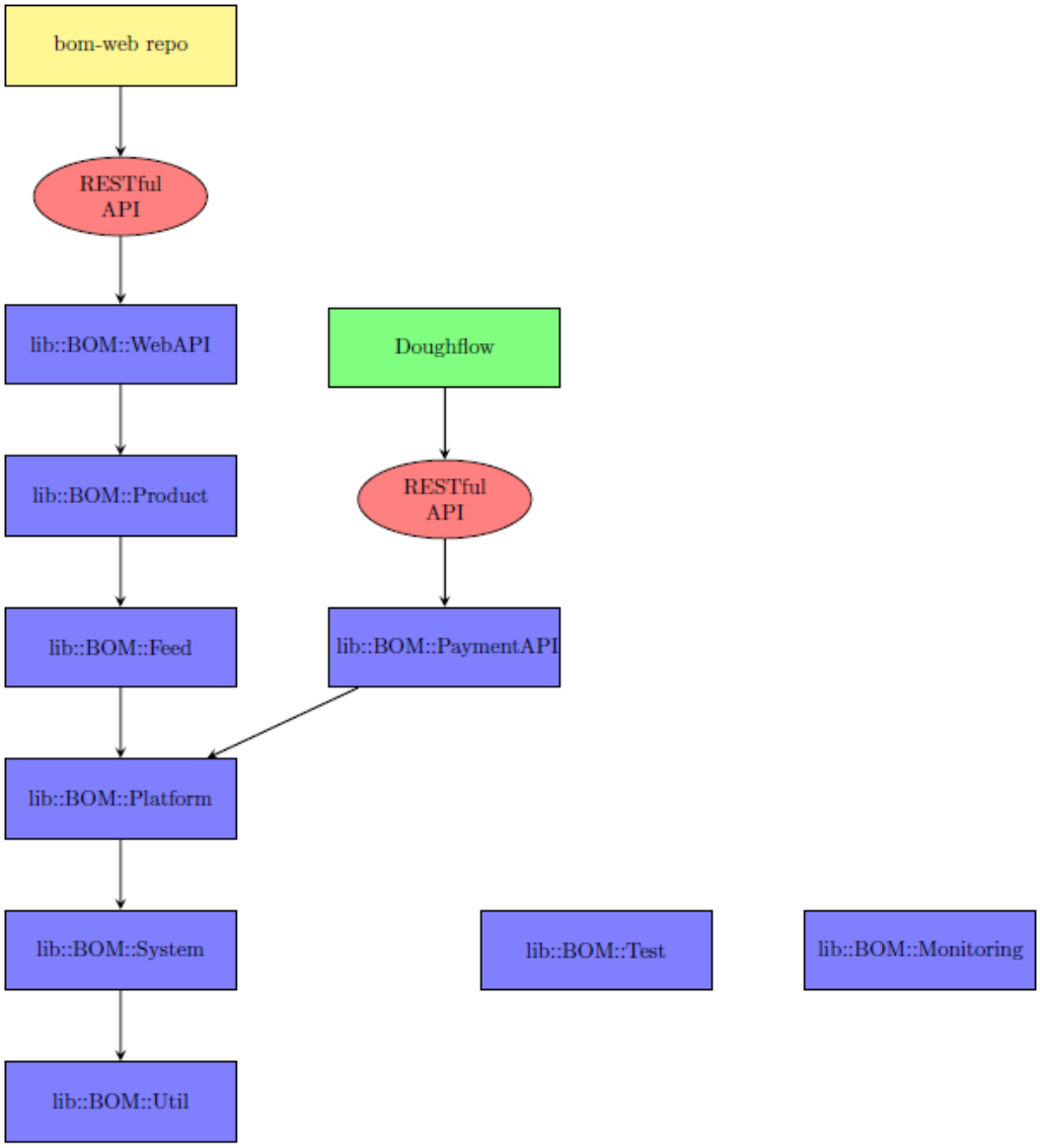
Chapter 12

lib

12.1 Summary

/lib is Betonmarkets' library of objects.

There are different **sub-libraries** for each **component** of the system.



The arrows indicate the code layers. i.e.:

- lib::BOM::Util should not use any other library (except CPAN modules).
- lib::BOM::System should use only lib::BOM::Util.
- lib::BOM::Data should use only lib::BOM::System and lib::BOM::Util.
- lib::BOM::Platform should use only lib::BOM::Data, lib::BOM::System and lib::BOM::Util.

- etc.

Chapter 13

PaymentAPI

Doughflow Integration API

Added by nick, last edited by Calum Halcrow on Jun 13, 2011

BetOnMarkets API Integration for DoughFlow

This document explains how BetOnMarkets will implement the API methods described in the **DoughFlow Gaming System Integration** document dated July 20, 2010.

Overview

BetOnMarkets is implementing a REST-style service in order to provide DoughFlow with the API methods required for the BetOnMarkets-DoughFlow integration. This service will be strictly REST-based, which means that:

- Retrieval of records will be accomplished via GET requests.
- Creation of records (such as deposits or withdrawal) will be accomplished via POST requests. As is typically the case with REST methods, a common workflow will have the BetOnMarkets API server redirect (via a 201 HTTP status code) to a GET URL after a record is created.
- Failed requests will be reported via HTTP status code, although perhaps with explanatory text in the HTTP message body.
- All requests will use a HTTP header-based authentication method described in the section titled "Request Authentication"

Request Authentication

To accomplish API requests against the BetOnMarkets API server, DoughFlow will connect via HTTPS to a specified server endpoint. This request must include a custom HTTP header called X-BOM-DoughFlow-Authorization which is computed as follows:

```
timestamp := current epoch timestamp, determined by the POSIX time() function
secret    := the string literal 'N73X49dS6SmX9Tf4'
conjoined := timestamp + secret (concatenated)
hash      := last 10 characters of hexadecimal MD5 hash of conjoined
header    := timestamp + ':' + hash
```

For example, if the current GMT epoch time is 1288156003, we compute:

```
conjoined = 1288156003N73X49dS6SmX9Tf4
full hash of conjoined = 72f76fdf7a28793c883d0c79a1780a4a
hash to use = 79a1780a4a
```

and then we send the header

```
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a
```

By rule, this header will only be accepted for authentication and authorization purposes if (a) the hash is computed correctly and (b) the provided timestamp differs by no more than 15 seconds from the timestamp on the BetOnMarkets API server. The purpose of this restriction is to minimize the window for potential replay attacks against the BetOnMarkets API server.

Note that in production, access to the BetOnMarkets API server will also be restricted by IP address. Therefore if this algorithm for computing the X-BOM-DoughFlow-Authorization header is compromised, it will be impossible for 'black hat' agents to make unauthorized transactions in the BetOnMarkets system without also compromising the production DoughFlow installation maintained jointly by BetOnMarkets and DoughFlow.

At a later time, BetOnMarkets may opt to replace the authentication scheme described above with an OAuth 1.0a or OAuth 2.0 scheme. In this case, sufficient lead time will be provided to ensure a smooth transition between old and new authentication protocols.

Client / Customer Authentication

When a BOM Client visits DoughFlow, we will generate a temporary 'handoff token' that will be passed to DoughFlow to begin the customer's DoughFlow session. That handoff token can be authenticated using the AuthCust (/session/validate) API described below. This allows DoughFlow to decide whether the Client in question is a valid BOM Client, and is only needed when initiating interactive sessions in the DoughFlow cashier.

Although customer's interactive session will likely result in other, later API calls to the Deposit/ProcessPayout/ReversePayout methods, these methods do not need to present the customer's session token in order to be validated.

Serialization Format

All of the REST methods described below accept multiple serialization formats. Any data returned by these methods will be serialized in a format determined from the original API request, with the following order of evaluation:

Content-Type: If the original request contains a Content-Type header, this will be the serialization format of the response.

The 'content-type' query parameter: For GET requests, if a query string parameter named 'content-type' (no quotes) is provided, it will be interpreted as if it were (but in lower precedence than) a Content-Type header.

Accept header evaluation: If a Content-Type or 'content-type' is not specified, the request will be examined for an 'Accept' header. Since an 'Accept' header can specify a series of acceptable content types in decreasing order of preference, the BOM API server will iterate through the types listed in the 'Accept' header until a valid format is found, or until no formats remain in the 'Accept' header.

Valid values for serialization are: 'application/json', 'text/xml', 'text/html', and 'text/x-yaml'. The default serialization format is 'application/json'. This will be used if no other format is specified.

Specifying an unsupported serialization format will cause the API request to fail.

Individual Methods

CustomerSignup: Will not implement. BetOnMarkets handles its own customer signups.

AuthCust: DoughFlow will perform the following GET request against the API server endpoint provided:

```
GET /session/validate?
client_loginid=CR1234&token=2902d064fd9842eeca4ab88b4bc29addabc04775 HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a
```

The resulting output will then be:

```
HTTP/1.0 200 Accepted
Content-Type: text/xml
...

<opt>
  <data details="https://api.betonmarkets.com/client/?loginid=CR1234"
status="accepted" />
</opt>
```

Any 200 OK status will denote a valid Client ID and Session ID pair. At this point the provided 'details' attribute could be chased to validate or update the user's current profile settings, although strictly speaking this is not necessary.

Any errors will result in appropriate error codes, 4xx for Authorization, 500 for Server Problems, etc.

GetBalance: DoughFlow will perform the following GET request against the API server endpoint provided:

```
GET /account/?client_loginid=CR1234&currency_code=USD HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a
```

The resulting output will then be:

```
HTTP/1.0 200 OK
Content-Type: text/xml
...

<opt>
  <data balance="1325.23" client_loginid="CR6426" currency_code="USD"
limit="35000.00" />
</opt>
```

From this output it should be easy to ascertain that the client's USD balance is $12.34.

Deposit: DoughFlow will perform the following POST request against the API server in order to deposit cash into a customer account:

```
POST /transaction/payment/doughflow/deposit HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a

<deposit
    amount="50.19"
    bonus="5.00"
    client_loginid="CR1234"
    created_by="INTERNET"
    currency_code="USD"
    fee="2.50"
    ip_address="1.2.3.4"
    payment_processor="NETeller"
    promo_id="promoX"
    trace_id="1234567890"
    transaction_id="TXN1029384756" />
```

The resulting response will be one of the following:

- A 201 CREATED response, pointing to the (newly created) transaction

```
HTTP/1.0 201 Created
Location: https://api.betonmarkets.com/transaction/payment/doughflow/record/?
client_loginid=CR1234&currency_code=USD&reference_number=12345
...
```

- An authorization error (4xx) reported via HTTP:

```
HTTP/1.0 401 Authorization Required
Content-Type: text/plain
...

Your request could not be processed due to authorization failure.
Please contact BetOnMarkets for assistance.
```

- A server error (5xx) reported via HTTP:

```
HTTP/1.0 500 Internal Server Error
Content-Type: text/plain
X-BOM-Incident-Token: a9b3c19d94b11a43
...

Your request could not be processed due to an internal server error.
Please contact BetOnMarkets and reference the incident ID a9b3c19d94b11a43 for
assistance with this issue.
```

ProcessPayout: DoughFlow will perform the following POST request against the API server in order to withdraw cash from a customer account:

```
POST /transaction/payment/doughflow/withdrawal HTTP/1.1
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a

<withdrawal
    amount="5500.67"
    client_loginid="CR1234"
    created_by="INTERNET"
    currency_code="USD"
    fee="2.50"
    ip_address="1.2.3.4"
    payment_processor="NETeller"
    trace_id="9876543210" />
```

The resulting response will handled identically to the **Deposit** case, with one exception. If the customer's balance is too low to allow the requested withdrawal, the following response will be provided:

```
HTTP/1.0 400 Bad Request
Content-Type: text/plain
...
```

```
Your request could not be processed for the following reason:
Invalid request: Requested withdrawal amount USD 5500.67 exceeds client balance USD
216.14
Please contact BetOnMarkets for assistance.
```

CreatePayout: This method will not be implemented by BetOnMarkets.

CancelPayout: This method will not be implemented by BetOnMarkets.

ReversePayout: DoughFlow will perform the following POST request against the API server in order to reverse a withdrawal of cash from a customer account:

```
POST /transaction/payment/doughflow/withdrawal_reversal HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a

<withdrawal_reversal
    amount="67.00"
    client_loginid="CR1234"
    created_by="INTERNET"
    currency_code="USD"
    ip_address="1.2.3.4"
    payment_processor="NETeller"
    trace_id="9876543210" />
```

Note that the trace_id of the withdrawal reversal request must match the trace_id of the original withdrawal request.

PreValidateDeposit: DoughFlow will perform the following GET request against the API server endpoint provided:

```
GET /transaction/payment/doughflow/deposit_validate?
amount=50.19&bonus=5.00&created_by=INTERNET&client_loginid=CR1234&currency_code=USD&fee=
HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a
```

Note that the parameters are to be exactly the same as when making an actual Deposit (but without trace_id), but must be submitted via a GET request rather than a POST.

Resulting output indicating that the deposit is allowed will then be:

```
HTTP/1.0 200 OK
Content-Type: text/xml
...

<opt>
  <data allowed="1" />
</opt>
```

If the deposit is not allowed, the resulting output will be similar to:

```
HTTP/1.0 200 OK
Content-Type: text/xml
...

<opt>
  <data allowed="0" message="Reason for not being allowed" />
</opt>
```

The 'message' field will be a message intended for the user of the API (i.e. not the end user) indicating why the deposit is not allowed.

Any errors will result in appropriate error codes, 4xx for Authorization, 500 for Server Problems, etc.

PreValidateWithdrawal: DoughFlow will perform the following GET request against the API server endpoint provided:

```
GET /transaction/payment/doughflow/withdrawal_validate?
amount=5500.67&client_loginid=CR1234&created_by=INTERNET&currency_code=USD&fee=2.50&ip_a
HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a
```

As with PreValidateDeposit, parameters are exactly the same as a Withdrawal (but without trace_id), but submitted via a GET request.

Resulting output is again similar to PreValidateDeposit:

```
HTTP/1.0 200 OK
Content-Type: text/xml
...

<opt>
  <data allowed="1" />
</opt>
```

or:

```
HTTP/1.0 200 OK
Content-Type: text/xml
...

<opt>
  <data allowed="0" message="Reason for not being allowed" />
</opt>
```

Any errors will result in appropriate error codes, 4xx for Authorization, 500 for Server Problems, etc.

AddressDiff: DoughFlow will perform the following POST request against the API server endpoint provided:

```
POST /client/address_diff/ HTTP/1.0
Host: api.betonmarkets.com
Content-Type: text/xml
X-BOM-DoughFlow-Authorization=1288156003:79a1780a4a

<address_diff
    client_loginid="CR1234"
    city="Paris"
    street="1 Rue Fromage"
    province="Francy"
    pcode="78457"
    country="FR"/>
```

Resulting output is just:

```
HTTP/1.0 200 OK
Content-Type: text/xml
...
```

Any errors will result in appropriate error codes, 4xx for Authorization, 500 for Server Problems, etc.

Labels: None

1 Comment

Calum Halcrow Oct 26, 2011

I think we should adjust this document so that it is Cashier agnostic then rename it to something like "BOM Cashier API Documentation".

Chapter 14

WebAPI

BetOnMarkets API Reference
Version 1

Note: see Oanda API for comparison
https://github.com/oanda/apidocs/blob/master/sections/reference.md

Terminology

Market	forex / indices / commodities / stocks / random
Underlying	frxUSDJPY / FCHI / R_75 / etc.
loginID	MLT29556, etc.
Contract	A bet
Strike(s), duration, payout	Parameters of a contract

REST best practices

- Use nouns in the URIs, not verbs. Resources are “things”, not “actions”.
- Use the slash ‘/’ in a URI to represent a parent-child, whole-part relationship.
- Put optional parameters in the query string (‘?xxx=yyy’).
- Use GET's for retrieving data, POST's to create new or update

Underlying Price API

See https://github.com/oanda/apidocs/blob/master/sections/rates.md

Endpoint	Description
GET /markets	Markets Discovery
GET /markets/underlyings	Underlying Discovery
GET /markets/:market/price	Get current price for all underlyings of class ‘:market’
GET /underlyings/:underlying/price	Get current price for single underlying
GET /underlyings/:underlying/candles	Get candlesticks for a single underlying

Bet Form API

With the API functionality below, you should be able to construct a bet form.

Endpoint	Description
GET /markets/:market/contracttypes	Discover available bet types for given market (rise/fall, higher/lower, onetouch, notouch etc). [For example, for UK stocks, only rise/fall bets are available.]
GET /markets/:market/contracttypes/underlyings	Discover underlyings for given bet type and market
GET /markets/:market/contracttypes/underlyings/:underlying/contractparams	Discover number of barriers (0,1,2), acceptable range for each barrier, available duration types (seconds, minutes, days), min and max durations for each duration type, available start times (for rise/fall bets), available payout currencies (USD, EUR, GBP, AUD), and min and max payout for each payout currency.

Bet Price API

With the API functionality below, you should be able to price and trade a bet.

Endpoint	Description
GET /contracttypes/:contracttype/:underlying/durationtypes/:durationtype/:duration/:payoutcurrency/:barrier1/:barrier2/:starttime	Get price of a bet (and its opposite) for a payout of 1 Optional param: time at which to price the bet (for BPOT functionality)
POST /contracttypes/:contracttype/:underlying/durationtypes/:durationtype/:duration/:payoutcurrency/:barrier1/:barrier2/:starttime	Buy a bet – query params: payout, limit price
POST /contract/:contractid	Close out specific bet that I have on my portfolio – query param: limit price

Accounts API

Endpoint	Description
GET /portfolio	Get portfolio (includes all account currencies)
GET /statement/:daysback	Get statement
GET /profittable/:daysback	Get profit table
GET /contract/:contractid	Get details about specific bet
GET /user	Get account information and settings (name, email, address, settings etc)